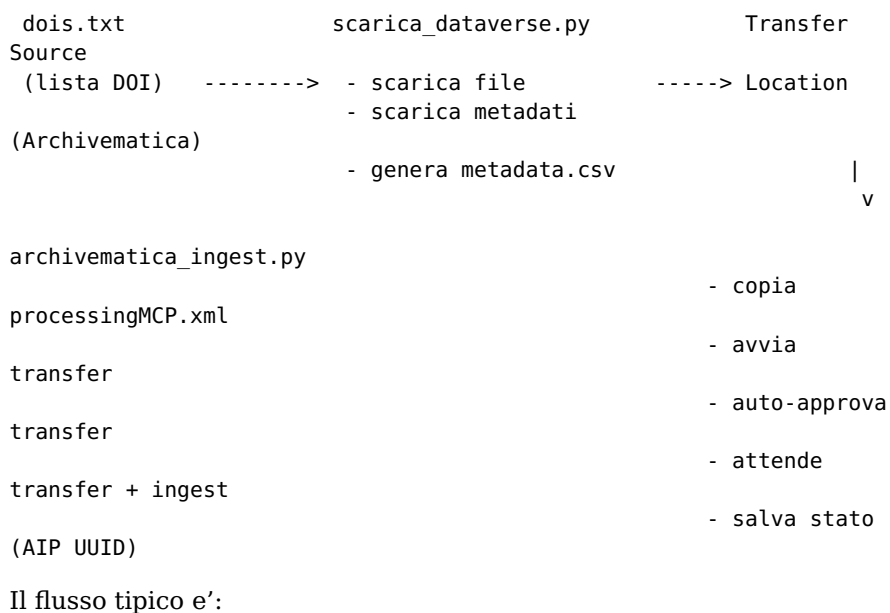




1. Si prepara `dois.txt` con l'elenco dei DOI da archiviare.
 2. Si esegue `scarica_dataverse.py`, che scarica tutte le versioni di ciascun dataset direttamente nella **Transfer Source Location** di Archivematica.
 3. Si esegue `archivematica_ingest.py`, che avvia il transfer e l'ingest per ogni pacchetto non ancora processato.
 4. Lo stato di ogni pacchetto (UUID di transfer, SIP, AIP) viene salvato in `stato_archivematica.json`, così le esecuzioni successive elaborano solo i dataset nuovi o non completati.
-

2. Prerequisiti

Software

```
pip install requests --break-system-packages
```

Accesso a Dataverse

- Una **API key Dataverse** (necessaria solo per dataset ad accesso ristretto; per dataset pubblici può essere lasciata vuota).

Accesso ad Archivematica

- **Dashboard API key** dell'utente Archivematica (es. `zfed`)
- **Storage Service API key** dell'utente Storage Service (es. `archivematica`)
- **UUID della Transfer Source Location** (vedi sotto come recuperarlo)
- Gli script devono avere accesso in lettura/scrittura alla Transfer Source Location sul filesystem

Permessi filesystem

L'utente che esegue gli script deve poter:

- **leggere e scrivere** nella Transfer Source Location
- **leggere** la cartella delle processing configuration:
`/var/archivematica/sharedDirectory/sharedMicroServiceTasksConfigs/processingMCPConfigs/`

Se necessario, aggiungere l'utente al gruppo `archivematica`:

```
sudo usermod -aG archivematica <utente>
# Richiede logout/login per avere effetto
```

Note su SSL

Archivematica è tipicamente raggiungibile via HTTPS anche in installazioni locali. Se usa un certificato self-signed, impostare `SSL_VERIFY=false` nel `.env` (o usare `--no-verify-ssl` da riga di comando).

Gli URL corretti per le API sono: - Dashboard: `https://<host>` (porta 443) - Storage Service: `https://<host>:8000` (porta 8000, con redirect HTTP a HTTPS)

3. Configurazione — file `.env`

Tutti i parametri di configurazione vanno nel file `.env` nella stessa cartella degli script. Il file viene caricato automaticamente all'avvio senza dipendenze esterne.

Creare il file .env

```
cp .env.template .env
nano .env
```

Contenuto del .env

```
# Dataverse
DATAVERSE_API_KEY=<chiave_dataverse>

# Archivematica Dashboard
AM_URL=https://192.168.139.39
AM_USER=zfed
AM_API_KEY=<chiave_dashboard>

# Archivematica Storage Service
SS_URL=https://192.168.139.39:8000
SS_USER=archivematica
SS_API_KEY=<chiave_storage_service>

# Transfer
AM_TRANSFER_SOURCE_UUID=<uuid_transfer_source>
AM_TRANSFER_TYPE=standard
AM_PROCESSING_CONFIG=dataverse_001

# Approva automaticamente il transfer senza intervento manuale
AM_AUTO_APPROVE=true

# SSL: false per certificati self-signed
SSL_VERIFY=false
```

Recuperare i valori mancanti

UUID della Transfer Source Location:

```
python3 archivematica_ingest.py --no-verify-ssl --list-sources
```

Processing configuration disponibili:

```
python3 archivematica_ingest.py --no-verify-ssl --list-processing-
configs
```

Errori comuni nel .env

Errore	Sintomo	Correzione
SS_URL=https://...	MissingSchema	Aggiungere i due punti: https://
SS_URL=http://...	302 Found	Usare https://
Chiavi API vuote	401 Unauthorized	Compilare AM_API_KEY e SS_API_KEY
UUID errato	404 Not Found	Usare --list-sources per trovare l'UUID corretto
Variabili esportate in sessione precedente	Valori vecchi ignorati	Aprire una nuova sessione shell

Priorita' delle variabili

1. Argomento da riga di comando (es. --ss-apikey)
2. Variabile d'ambiente di sistema (impostata con export)
3. File .env
4. Valore di default nel codice

Attenzione: se in una sessione precedente hai eseguito `export $(grep -v '^#' .env | xargs)`, le variabili rimangono attive e hanno priorit  sul `.env`. In caso di problemi aprire una nuova sessione shell.

Sicurezza

```
echo ".env" >> .gitignore
```

4. Script 1 — scarica_dataverse.py

Cosa fa

Per ogni DOI elencato in `dois.txt`:

1. Recupera **tutte le versioni** pubblicate del dataset
2. Per ciascuna versione:
 - scarica tutti i file del dataset
 - salva i metadati Dataverse in `dataverse.json`
 - genera un `metadata.csv` Dublin Core per quella versione
3. Genera un `metadata.csv` complessivo nella radice del pacchetto con i metadati Dublin Core di **tutte le versioni**, nel formato letto nativamente da Archivematica (METS dmdSec)

File di input: `dois.txt`

```
doi:10.13130/RD_UNIMI/KS2PUX
doi:10.13130/RD_UNIMI/NFURUT
# questo e' un commento, viene ignorato
doi:10.13130/RD_UNIMI/QBRJNN
```

Opzioni da riga di comando

Opzione	Default	Descrizione
<code>--dois <file></code>	<code>dois.txt</code>	File con l'elenco dei DOI
<code>--output <cartella></code>	<code>DATASET TRASFERITI</code>	Cartella di destinazione (Transfer Source Location)
<code>--apikey <chiave></code>	dal <code>.env</code>	API key Dataverse
<code>--no-verify-ssl</code>	<code>false</code>	Disabilita verifica certificato SSL

Esempi

```
# Con configurazione da .env
python3 scarica_dataverse.py

# Specificando l'output verso la Transfer Source di Archivematica
python3 scarica_dataverse.py \
  --output /mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE

# Con certificato self-signed
python3 scarica_dataverse.py --no-verify-ssl
```

Output a schermo

```
=====
Dataverse UNIMI -- Download automatico (tutte le versioni)
Sorgente DOI : dois.txt (3 DOI)
```

```
Destinazione : /mnt/.../TRANSFER_SOURCE
API key      : *****5678
SSL verify   : False
```

```
[1/3] DOI: doi:10.13130/RD_UNIMI/KS2PUX
--> Recupero lista versioni...
--> 1 versione/i trovata/e: v1.0
-- Versione v1.0 [RELEASED]
[metadata] dataverse.json salvato
[data] 4 file trovati, inizio download...
  Scarico: works.tab ... OK (25 KB)
  Scarico: geolocations.tab ... OK (1 KB)
  Scarico: collections.tab ... OK (2 KB)
  Scarico: README.txt ... OK (3 KB)
[metadata] metadata.csv (versione) generato (4 righe)
[metadata] metadata.csv generato (4 righe, 1 versione/i)
[validazione] metadata.csv OK -- nessun problema rilevato.
OK Stato: ok | Versioni: 1 | File OK: 4 | Errori file: 0
```

```
RIEPILOGO FINALE
Completati : 3
Parziali   : 0
Errori      : 0
```

Comportamento idempotente

- File già scaricati: **saltati**
 - dataverse.json e metadata.csv di versione: generati solo se mancanti
 - metadata.csv complessivo: **sempre rigenerato** per includere nuove versioni
-

5. Script 2 — archivematica_ingest.py

Cosa fa

Per ogni pacchetto DOI nella Transfer Source Location non ancora completato:

1. Copia processingMCP.xml nella radice del pacchetto
2. Avvia il **Transfer** con nome AIP-<YYYYMMDD>-<DOI>
3. Approva automaticamente il transfer tramite API (se AM_AUTO_APPROVE=true)
4. Attende il completamento del Transfer (polling)
5. Attende il completamento dell'**Ingest** (SIP -> AIP)
6. Salva l'UUID dell'AIP nel file di stato

Ogni cartella DOI viene trasferita come **un unico pacchetto** con tutte le sue versioni.

Come funziona l'auto-approve

Lo script interroga l'endpoint /api/transfer/unapproved/ per trovare i transfer in attesa di approvazione, identifica quello corrispondente al pacchetto corrente tramite il nome, e lo approva via /api/transfer/approve/. Questo elimina la necessità di cliccare manualmente "Approve transfer" nel Dashboard.

Opzioni da riga di comando

Sorgente e stato

Opzione	Default	Descrizione
--source <cartella>	da Transfer Source	Directory con i pacchetti
--state-file <file>	stato_archivematica.json	File JSON di tracciamento

Dashboard Archivematica

Opzione	Default	Descrizione
--am-url <url>	dal .env	URL del Dashboard
--am-user <utente>	dal .env	Utente Dashboard
--am-apikey <chiave>	dal .env	API key Dashboard

Storage Service

Opzione	Default	Descrizione
--ss-url <url>	dal .env	URL Storage Service
--ss-user <utente>	dal .env	Utente Storage Service
--ss-apikey <chiave>	dal .env	API key Storage Service

Transfer

Opzione	Default	Descrizione
--transfer-source <UUID>	dal .env	UUID Transfer Source Location
--transfer-type <tipo>	standard	Tipo di transfer
--processing-config <nome>	dataverse_001	Processing configuration
--auto-approve	dal .env	Approva automaticamente il transfer

Utility

Opzione	Descrizione
--list-sources	Elenca le Transfer Source Locations ed esce
--list-processing-configs	Elenca le processing configuration ed esce
--no-verify-ssl	Disabilita verifica certificato SSL
--poll-interval <sec>	Intervallo tra polling (default: 15s)
--dry-run	Mostra cosa verrebbe fatto senza eseguire

Esempi

```
# Esecuzione standard (tutto da .env, incluso auto-approve)
python3 archivematica_ingest.py

# Con certificato self-signed
python3 archivematica_ingest.py --no-verify-ssl

# Lista Transfer Source disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Lista processing configuration disponibili
```



```

v2.0/
  objects/
    works_v2.tab
  metadata/
    dataverse.json
    metadata.csv
  metadata/
    metadata.csv
versioni
METs dmdSec
  processingMCP.xml
transfer

```

<- Dublin Core di TUTTE le
letto da Archivematica ->
<- copiato prima del
transfer

Formato metadata.csv (radice del pacchetto)

filename	dc.title	dc.creator	dc.date
objects/v1.0/objects/works.tab	Titolo	Autore	2026-03-04
objects/v1.0/objects/README.txt	Titolo	Autore	2026-03-04
objects/v2.0/objects/works_v2.tab	Titolo	Autore	2026-03-04

Archivematica traspone questi metadati nel METS.xml come <dmdSec MDTYPE="DC">.

7. File di stato e ripresa dopo interruzioni

stato_archivematica.json

```

{
  "doi_10.13130_RD_UNIMI_KS2PUX": {
    "transfer_name": "AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX",
    "transfer_uuid": "bde477ff-...",
    "sip_uuid": "a1b2c3d4-...",
    "aip_uuid": "e5f6a7b8-...",
    "status": "ingested",
    "completed_at": "2026-06-19T10:42:13+00:00",
    "error": null
  }
}

```

Valori di status

Valore	Significato
in_progress	Avviato ma non completato
transferred	Transfer ok, ingest non ancora completato
ingested	Completato – saltato nelle esecuzioni successive
failed	Errore – verra' ritentato da zero al prossimo avvio

Reset manuale di un pacchetto

```

python3 -c "
import json
s = json.load(open('stato_archivematica.json'))
s.pop('doi_10.13130_RD_UNIMI_KS2PUX', None)
json.dump(s, open('stato_archivematica.json', 'w'), indent=2)
print('Rimosso')
"

```


Reset completo

```
echo '{}' > stato_archivematica.json
```

8. Risoluzione dei problemi comuni

MissingSchema / URL non valido

MissingSchema: Invalid URL 'https://...'

Mancano i due punti: https:// non https//.

302 Found sulla porta 8000

Nginx reindirizza HTTP a HTTPS. Usare https://:

```
SS_URL=https://192.168.139.39:8000
```

401 Unauthorized

Le API key sono vuote o errate. Verificare AM_API_KEY e SS_API_KEY nel .env.

404 Not Found sulla Transfer Source UUID

L'UUID nel .env non esiste su questo server. Recuperare quello corretto:

```
python3 archivematica_ingest.py --no-verify-ssl --list-sources
```

SSL Certificate Verify Failed

SSLCertVerificationError: certificate verify failed: self-signed certificate

Aggiungere SSL_VERIFY=false nel .env oppure usare --no-verify-ssl.

Permission Denied su processingMCPConfigs

PermissionError: [Errno 13] Permission denied

```
sudo usermod -aG archivematica <utente>  
# Poi fare logout e login
```

Transfer REJECTED al riavvio dello script

Se un transfer precedente era stato rifiutato, lo script lo rileva automaticamente (status failed) e riparte da zero senza bisogno di cancellare manualmente il file di stato.

400 "Unable to determine the status of the unit"

Errore transitorio: Archivematica non ha ancora registrato il transfer. Lo script ritenta automaticamente 2 volte con 20 secondi di attesa.

Auto-approve non funziona

Se [approve] non trova il transfer in attesa, verificare:

1. Che AM_AUTO_APPROVE=true sia nel .env
2. Che il transfer sia effettivamente in stato USER_INPUT nel Dashboard

3. Aumentare APPROVE_TRANSFER_WAIT nel codice (default: 10s) se Archivematica e' lenta ad avviare il transfer

Elasticsearch non raggiungibile

```
elastic_transport.ConnectionError: Connection refused ...  
127.0.0.1:9200
```

```
sudo systemctl start elasticsearch  
curl -s http://localhost:9200
```

Transfer non appare nel Dashboard

```
ls  
/var/archivematica/sharedDirectory/watchedDirectories/activeTransfers/standardTransfer/
```

```
sudo systemctl status archivematica-mcp-server archivematica-mcp-  
client
```

9. Esempio di esecuzione completa

Configurazione iniziale (una tantum)

```
# Clona il repository  
git clone https://github.com/zfed/Arkive.git  
cd Arkive/tools/dataverse-archivematica  
  
# Installa dipendenze  
pip install requests --break-system-packages  
  
# Crea il file .env  
cp .env.template .env  
nano .env  
  
# Recupera l'UUID della Transfer Source  
python3 archivematica_ingest.py --no-verify-ssl --list-sources  
# Aggiorna AM_TRANSFER_SOURCE_UUID nel .env  
  
# Verifica le processing configuration disponibili  
python3 archivematica_ingest.py --no-verify-ssl --list-processing-  
configs  
# Aggiorna AM_PROCESSING_CONFIG nel .env se necessario
```

Esecuzione

```
# 1. Prepara l'elenco dei DOI  
cat > dois.txt << 'EOF'  
doi:10.13130/RD_UNIMI/KS2PUX  
doi:10.13130/RD_UNIMI/NFURUT  
doi:10.13130/RD_UNIMI/QBRJNN  
EOF  
  
# 2. Scarica i dataset nella Transfer Source  
python3 scarica_dataverse.py \  
--output /mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE  
  
# 3. Trasferisci e ingesta in Archivematica (completamente  
automatico)  
python3 archivematica_ingest.py  
  
# 4. Verifica lo stato  
cat stato_archivematica.json | python3 -m json.tool
```

Per le esecuzioni successive, ripetere i passi 2 e 3: gli script elaborano solo cio' che e' nuovo o non ancora completato.

